

Bayesian Mixed Effect Models

There are a number of drawbacks to likelihood-based estimation of mixed effect models. We have seen in the previous two chapters that inference is often difficult. Furthermore, we may encounter problems even in the maximization of the likelihood leading to untrustworthy results. This leads us to explore alternative approaches to inference for these models.

Bayesian methods for inference are well established. In the past, fitting Bayesian methods had been difficult in comparison to the often one-line R command used to fit likelihood-based models. The `glm` and `lmer` commands in R are powerful in their scope. In contrast, fitting Bayesian models required more effort in setting up and implementation. In recent years, Bayesian modeling has become more accessible. It still requires more effort but one does not have to be an expert to make some progress.

It is important to understand that Bayesian inference is not simply a replacement for classical (Frequentist) inference. We can equally well address the substantive issues of the specific application but the questions and answers will be somewhat different. In particular, the classical theory of hypothesis testing does not fit well with the Bayesian approach and so it would be a mistake to expect the exact equivalence of p -values and the like. The answers will come in a different form. Historically, there have been purists who insist on an entirely Frequentist or Bayesian solution, but now many would recognize that both methods have strengths and weaknesses and one can learn much by using both.

A full explanation of Bayesian methods would take a whole book — I recommend Gelman et al. (2013) as a good starting point. Here we shall simply give a sufficient overview so that we can get started on analyses of the types of data we have seen in the previous two chapters. Likelihood-based methods as used by the `lme4` package depend on the likelihood $l(\theta|\mathbf{y})$ for parameters θ and data $\mathbf{y}$. We discussed how this likelihood can be constructed in Chapter 10. We can maximize this likelihood to produce maximum likelihood estimates and use the general theory of likelihood to construct hypothesis tests and confidence intervals. We considered the parameters θ to be fixed (but unknown) and the data $\mathbf{y}$ to be random. The Bayes approach considers the θ to be random but the data $\mathbf{y}$, once it has been observed, to be fixed. We have, from Bayes theorem:

$$p(\theta|\mathbf{y}) \propto l(\theta|\mathbf{y})p(\theta)$$

The term $p(\theta|\mathbf{y})$ is called the posterior probability (or density for continuous $\mathbf{y}$). This expresses our knowledge about the parameter values after seeing the data. We can use this probability to make decisions or predictions. The price of getting a prob-

ability on the parameters is that we must first specify a prior probability, $p(\theta)$, on the parameters. This should be done before looking at the data. Sometimes we know very little about these parameters. In such cases, we want an *uninformative* prior. In other cases, we may know rather more, in which case, the prior should reflect that knowledge.

The specification of the prior is the most important task that distinguishes Bayesian modeling. Many researchers feel uncomfortable about this choice because of the subjective choice involved. However, one should recognize that the Frequentist analysis is not entirely objective and deterministic either. The analyst often has to make strong assumptions to specify the model. Diagnostics can check some of these assumptions but not entirely or sometimes at all. One has to accept that there will be a subjective element to the analysis whichever method is used. It is best simply to be honest about this. In some cases, the prior enables us to use subject matter knowledge in the analysis more effectively. Even when we wish to assume as little as possible, there are some good ways to choose the prior to match this assumed lack of knowledge.

The formula linking the posterior to the likelihood and prior is rarely a means by which the posterior can be explicitly calculated. We demonstrate two distinct solutions in the chapter. The STAN language, described in Stan Development Team (2015), implements a simulation-based solution. In contrast, the INLA method, described in Rue et al. (2009), uses an approximation.

12.1 STAN

STAN is an example of a simulation-based approach to computing posterior densities. The idea is to set up a Markov chain of realizations of θ such that the equilibrium distribution is the posterior distribution. These methods aim to give you a sample from the posterior distribution that you can use to answer your questions. This class of methods is known as MCMC (Monte Carlo Markov chain). We do not explain how MCMC works. The reader is referred to Gelman et al. (2013) or other texts. Here we simply present the implementation by means of examples.

The first major general software for Bayesian computing was BUGS (Bayesian inference Using Gibbs Sampling) introduced in 1989. The book by Lunn et al. (2012) provides a general introduction. BUGS developed into WinBUGS and more recently OpenBUGS. JAGS is another offshoot of the original BUGS. Around 2012, STAN became available and is described in Stan Development Team (2015). The languages used in all these software packages are quite similar although underneath there are substantial differences in the implementation. I have chosen STAN here because it was originally motivated by the fitting of just the types of models we are interested in. It is much more general but it does work particularly well for our applications. Another reason is that STAN is faster for our particular problems. Even so, this is not a strong preference and one could very reasonably choose one of the other options.

STAN will require us to venture beyond R a little, but we will be able to prepare the data, call the fitting method and process the results in R by means of the `RStan` package. There are R packages which provide an entirely R-based MCMC solution. For example, the `MCMCglmm` package of Hadfield (2010) offers some flexibility

in fitting Bayesian models to grouped, hierarchical and longitudinal data. However, it is inevitably limited in the functionality it provides. Of course, R is a very programmable language so it is entirely possible to implement methods to fit virtually any model. However, there are reasons beyond simply the programming effort required why this would be less than effective. Fitting Bayesian models using MCMC or related techniques is inherently computationally intensive. R has many virtues but speed is not its strongest point. This leads one to go beyond R in search of a language for describing Bayesian models that will be both more efficient and also convenient for the specification of Bayesian models.

Before proceeding, you will need to install STAN as described on the STAN website at mc-stan.org. This is more work than simply installing the R package, and the details will vary according to the operating system you are running. STAN derives its speed from translating its code into C++, then compiling and running this. This means you will need a C++ compiler (but no knowledge of C++).

One-Way ANOVA: Let's see how we can use STAN to fit a Bayesian model to the `pulp` data that we have already explored with `lme4` in Chapter 10. See Figure 10.1 for a plot of the data. The model has a single factor at a levels:

$$y_{ij} = \mu + \alpha_i + \epsilon_{ij} \quad i = 1, \dots, a \quad j = 1, \dots, n_i$$

where the α s and ϵ s are independently distributed $N(0, \sigma_\alpha^2)$ and $N(0, \sigma_\epsilon^2)$, respectively. There is a single fixed effect parameter μ and two random effect parameters σ_α^2 and σ_ϵ^2 . We must specify prior distributions for these three parameters.

Here is the STAN code we have used to fit our model. It could be used for any one-way ANOVA data. Let's suppose we have N observations in total with J groups with a response and predictor named just so. You don't need to understand the STAN code to follow the output so you can skip to the discussion of the output unless you are interested in writing your own STAN programs. You should copy this code into a file called `oneway.stan` and save it into the working directory for your R session.

```
data {
  int<lower=0> N;
  int<lower=0> J;
  int<lower=1, upper=J> predictor[N];
  vector[N] response;
}
parameters {
  vector[J] eta;
  real mu;
  real<lower=0> sigmaalpha;
  real<lower=0> sigmaepsilon;
}
transformed parameters {
  vector[J] a;
  vector[N] yhat;

  a <- mu + sigmaalpha * eta;

  for (i in 1:N)
    yhat[i] <- a[predictor[i]];
```

```

}
model {
  eta ~ normal(0, 1);

  response ~ normal(yhat, sigmaepsilon);
}

```

The first block of code describes the format of the data. C++ is a much fussier language than R regarding the declaration of variables and the types of variables that are allowed. This means we often have to do a little work to prepare the data in a format that the STAN code will allow. In this case, we need numerical values for the factors rather than alphanumeric strings. We specify N , J , the response and the predictor. We need to tell it the length of the two vectors (which is N). We can also specify bounds on the values between the angle brackets. Here we have required that N and J be strictly positive and that the predictor values lie in the set $\{1, \dots, J\}$. These bounds are not essential but it is a good idea to put them in as it will catch some forms of invalid data that might be supplied.

The next block contains the parameters. The μ , σ_α and σ_ϵ parameters correspond to the μ , σ_α and σ_ϵ parameters used in previous modeling. The η parameter (actually a vector of parameters) will be needed in the construction of the model. We have specified lower bounds of zero on σ_α and σ_ϵ . This is essential to the proper treatment of these parameters.

The transformed parameters block combines the previously declared parameters in preparation for declaring the model in the next block. The a vector corresponds to the random effects declared previously although this includes the mean term μ . The mean values of the response are given by $yhat$ and are just the appropriate value from the a vector.

The final model block defines the model. Here we declare the η variables to be standard normal. We then specify the response to be normally distributed with mean and standard deviation σ_ϵ (not the variance!).

You may have noticed that we have not explicitly specified the priors for the three parameters: μ , σ_α and σ_ϵ . By default, improper uniform priors will be declared. For μ , this is the whole real line. Because both limits have not been specified, this is not a proper uniform distribution—that is why it is called *improper*. For the two standard deviation parameters, the prior is uniform on the positive real line. This explains the necessity of specifying bounds on the parameters earlier so that appropriate priors will be used. Our intent in choosing these particular priors is to express our lack of knowledge about these parameters.

We need to arrange the data in R into a list format consistent with the data statement of the STAN program:

```

data(pulp, package="faraway")
pulpdat <- list(N=20, J=4, response=pulp$bright, predictor=as.numeric(
  ↪ pulp$operator))

```

We load the library that interfaces STAN to R:

```
library(rstan)
```

When loading the library, you may receive a prompt about how to use the multiple cores in your CPU. It is worth taking this advice as it will speed up the computations significantly. Now we can fit the STAN model as follows:

```
fit <- stan(file = "oneway.stan", data = pulpdats)
```

This assumes that the `oneway.stan` file containing the STAN code is in the working directory used by R. This command involves three major steps. The first is to translate the STAN code into C++. If you have made a syntax error in your STAN code, it will be revealed here. The second step is to compile the C++ code. We might change the data or simply want to run the MCMC again with different settings but this would not require repeating these two steps. There is a way to split the process into three separate steps if this would save some time. The third step actually runs the MCMC. A large amount of output will be produced, most of which can be ignored. Compiler warnings are usually not important. There are typically some cryptic warnings regarding the progress of the Markov chains. These can be ignored provided there are not too many. The entire process might take a few minutes depending on the quality of your computer.

The first step is to examine the model fitting diagnostics as seen in Figure 12.1. These diagnostics are entirely distinct from usual residual-based diagnostics which check the adequacy of the model. We are checking whether the MCMC process has produced a satisfactory result. The Markov chain needs to start from some initial values which should be feasible values of the parameters but may well be unlikely values. This means it may take some time before the chain settles into anything resembling the equilibrium distribution. This initial period is called the *burn-in* or *warm-up* time. We can determine an appropriate length for this period by plotting the chain(s). STAN recognizes this potential sensitivity to initial values and, by default, computes four chains from different randomly chosen initial values. We start with the `mu` parameter:

```
traceplot(fit, pars="mu", inc_warmup = TRUE)
```

The plot, seen in the first panel of Figure 12.1, shows the four complete chains starting from the initial values. The greyed area shows the warm-up period. We can see that all four chains have stabilized well within this period. If this stabilization fails to occur within the period, we can increase the length of the warm-up period or perhaps give the program some hints about suitable initial values.

For the subsequent computations, STAN discards this warm-up half of the chain. The resolution of the first plot means that it is difficult to see the behavior of the chains once they become more stationary. This requires that we redo the plot without the warm-up period as seen in the second panel of Figure 12.1:

```
traceplot(fit, pars="mu", inc_warmup = FALSE)
```

The Markov chain, by its nature, is dependent. We can still use it to estimate the posterior density or functions of it, such as the mean. However, a positively dependent chain contains less information than an independent sample of the same size. The less dependent the chain, the better. A chain that is not strongly dependent is said to *mix* well. In less favorable situations, a chain may become “stuck” in some sub-region of the parameter space for long sequences. Such chains are said to mix poorly. The four chains shown in the second panel of Figure 12.1 exhibit strong mixing as they all vary randomly around a constant level with no obvious dependence.

If you find a poorly mixing chain, you can run the chain for longer. The default chain length is 2000 iterations and this can be substantially increased. For simpler

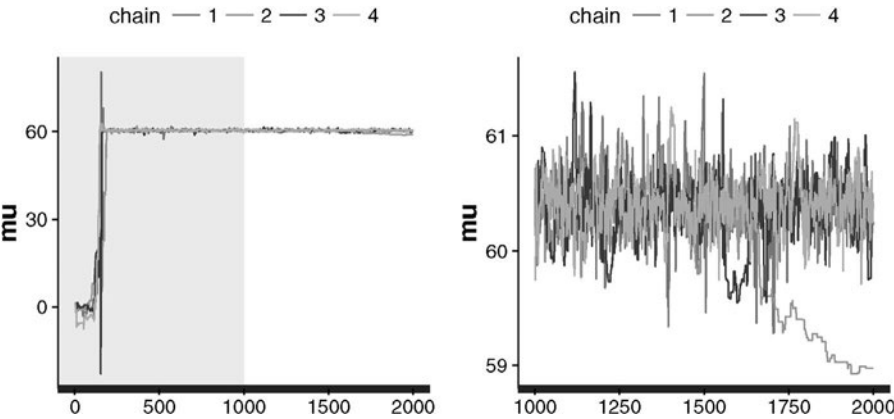


Figure 12.1 Diagnostic plots for μ from the STAN model for the pulp data. Version with a warm-up period is shown on the left. Four chains are shown in each case.

models, this additional computational time may not matter much. However, in more extreme cases, one may be concerned that equilibrium has not been reached despite the extra iterations and that the chain is unreliable. In such cases, one may need to tinker with the construction of the chain or even change the prior.

We can make similar plots for σ_{α} and σ_{ϵ} , which both show a satisfactory outcome. Now we can examine the printed output:

```
fit
4 chains, each with iter=2000; warmup=1000; thin=1;
post-warmup draws per chain=1000, total post-warmup draws=4000.

      mean se_mean  sd  2.5%  25%  50%  75%  97.5% n_eff Rhat
eta[1]    -0.33    0.02  0.65  -1.62 -0.74 -0.32  0.10  0.89   910 1.01
eta[2]    -0.70    0.02  0.70  -2.13 -1.15 -0.65 -0.23  0.55   999 1.00
eta[3]     0.43    0.03  0.69  -0.92 -0.03  0.42  0.89  1.82   562 1.02
eta[4]     0.57    0.03  0.69  -0.73  0.11  0.54  1.02  1.96   420 1.02
mu        60.42    0.03  0.32  59.77 60.26 60.40 60.54 61.39   124 1.05
sigmaalpha 0.52    0.04  0.49  0.06  0.23  0.37  0.61  2.14   129 1.03
sigmaepsilon 0.36    0.00  0.07  0.25  0.31  0.35  0.40  0.53  1085 1.00
a[1]       60.28    0.00  0.15  59.98 60.19 60.28 60.38 60.59  2526 1.00
a[2]       60.14    0.00  0.17  59.82 60.03 60.14 60.25 60.48  1413 1.00
a[3]       60.57    0.00  0.16  60.26 60.46 60.56 60.67 60.88  1485 1.00
a[4]       60.61    0.00  0.16  60.29 60.50 60.61 60.72 60.93  2042 1.00
yhat[1]    60.28    0.00  0.15  59.98 60.19 60.28 60.38 60.59  2526 1.00
yhat[2]    60.28    0.00  0.15  59.98 60.19 60.28 60.38 60.59  2526 1.00
...
lp_        7.80    0.12  2.81  1.35  6.11  8.27  9.91 12.03   582 1.00
```

For each parameter, n_{eff} is a crude measure of effective sample size, and R_{hat} is the potential scale reduction factor on split chains (at convergence, $R_{\text{hat}}=1$).

We have 2000 iterations for each of four chains but 1000 iterations are discarded for the first half of each chain, making 4000 iterations in all. If you use a much larger number of iterations, you may generate more posterior samples than convenient. You

can reduce this by thinning. STAN does not thin by default hence the `thin=1`. Setting `thin=10` would take every tenth observation and discard the rest.

For each posterior distribution, we have a dependent sample of size 4000. Various summary statistics are calculated as seen in the output. The samples are dependent but this is not a problem unless this is too strong. We can use the methods of time series analysis to compute various measures of dependence. The `Rhat` statistic is a measure of dependence. Values close to one indicate less dependence. The effective sample size tells us the equivalent independent sample size. These would be equal to the iteration total of 4000 for independent samples. We see that all these are well into the hundreds which is sufficient for most purposes.

The `eta` values represent scaled versions of the random effects. These are not particularly useful to us. The `mu` posterior mean is 60.4 just as in previous analyses. The `sigmaalpha` and `sigmaepsilon` posterior means are 0.52 and 0.36, respectively. We can also see how the posterior means for the four groups vary in the `a` parameters. The `yhat` posterior expected means for the response for each case just take the corresponding value of `a` depending on the group. For this reason, we have not printed all these out. The final `lp` term is the “log probability” and can be used to diagnose some problems with the chain.

The 2.5 and 97.5 percentiles form 95% credible intervals for the parameters. We claim a 0.95 probability that the parameter lies within the interval. This interpretation is different from a confidence interval where the 0.95 probability refers to the chance that the interval contains the parameter.

We can now examine the posterior distributions of σ_α and σ_ϵ :

```
library(reshape2)
postsig <- extract(fit, pars=c("sigmaalpha", "sigmaepsilon"))
ref <- melt(postsig, value.name="bright")
ggplot(data=ref, aes(x=bright, linetype=L1)) + geom_density() + xlim(0, 2)
  ↳ +scale_linetype(name="SD", labels=c("operator", "error"))
```

These can be seen in the first panel of Figure 12.2. We see that the posterior for the error SD is much more concentrated than the operator SD. The operator SD could be negligibly small but there is also a small chance that it could be substantially larger than the error SD.

In previous analysis of this data in Chapter 10, we tested a null hypothesis that there was no difference between the operators by setting $H_0 : \sigma_\alpha = 0$. The hypothesis testing formulation does not make sense within the Bayesian approach. Even so, we may still be interested whether there is a difference between the operators. We might formulate this question in terms of $P(\sigma_\alpha = 0)$. We have set a continuous prior and so we get a continuous posterior. This means the answer to our question is inevitably zero. For there to be a chance of a nonzero answer, we would need to specify a prior that gave strictly positive probability to the event that $\sigma_\alpha = 0$.

There are three reasons why we do not want to set such a prior. Firstly, for various technical reasons, the computation of the posterior resulting from such priors is difficult. Secondly, we would need to specify the prior probability that $P(\sigma_\alpha = 0)$. We may accept that the specification of a model requires some subjectivity but setting a probability like this feels uncomfortably close to fixing the conclusion. Thirdly, we might not view this as a sensible question. If the operators are four different people,

why would we expect them to behave identically? Instead, we might ask whether the difference between the operators is less than some specified small value. For example, we could compute $P(\sigma_\alpha < 0.1)$.

```
mean(postsig$sigmaalpha < 0.1)
```

```
[1] 0.05725
```

This is not a p -value. We have had to specify what we meant by “small,” but this is a judgement that we might reasonably ask of anyone with knowledge of the application. In this case, the response is recorded with only one figure after the decimal point so 0.1 is a reasonable choice, but choices will differ according to the situation. In this example, Figure 12.2 tells us that the difference between operators is likely to be considerable.

We may also be interested in the differences between specific operators. We can plot these posterior distributions:

```
opre <- rstan::extract(fit, pars="a")
ref <- melt(opre, value.name="bright")
ggplot(data=ref, aes(x=bright, linetype=factor(Var2)))+geom_density()+
  ↪ scale_linetype(name="operator", labels=LETTERS[1:4])
```

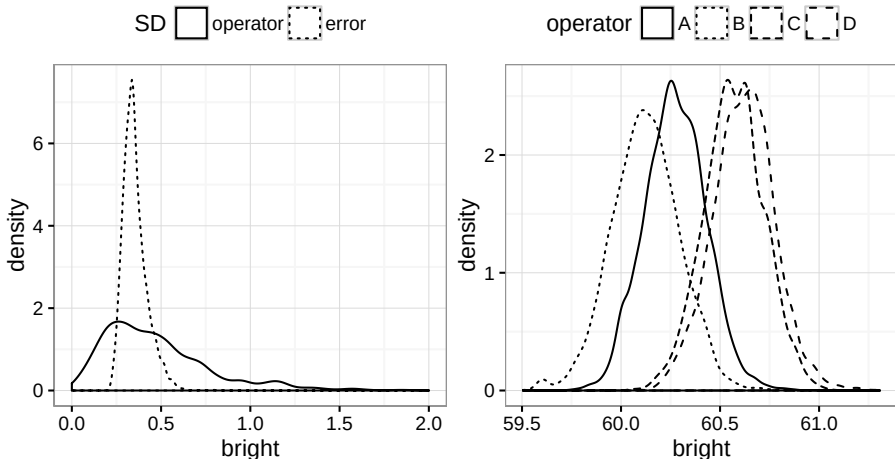


Figure 12.2 Posterior distributions for the standard deviations on the left and for the operator effects on the right.

We can see the posterior distributions for these effects in the second panel of Figure 12.2. We see that A and B tend to have lower responses than C and D but there is a substantial overlap such that we cannot be sure of any difference between the operators. We can compute a probability like $P(\alpha_A > \alpha_D)$ from the posterior samples:

```
mean(ref[,1] > ref[,4])
```

```
[1] 0.0665
```

We have used uninformative priors for the parameters but we should think more carefully about this choice. In particular, we would have good reason to expect that there would not be extreme differences between and within operators. For this reason we might specify a prior that puts most mass on small to moderate values but does not completely exclude the possibility of much larger values. A good distribution

for this is the Cauchy as the Gaussian (or similar) distributions have light tails and do not allow any reasonable possibility of extremes. In contrast, the Cauchy does have heavy tails but will give most weight to the small/moderate values that we most anticipate. For strictly positive parameters like standard deviations, we can use the half-Cauchy.

We modify the `oneway.stan` file to add two new lines to the model block:

```
sigmaalpha ~ cauchy(0, 1);
sigmaepsilon ~ cauchy(0, 1);
```

We call the modified file `onewaycauchy.stan` and rerun it:

```
fit <- stan(file = "onewaycauchy.stan", data = pulpdatt)
```

We can now examine the output:

```
print(fit, pars=c("mu", "sigmaalpha", "sigmaepsilon", "a"))
```

	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
mu	60.38	0.04	0.27	59.65	60.26	60.40	60.52	60.86	47	1.10
sigmaalpha	0.43	0.03	0.28	0.06	0.23	0.35	0.55	1.11	71	1.05
sigmaepsilon	0.35	0.00	0.07	0.25	0.30	0.34	0.39	0.52	381	1.01
a[1]	60.28	0.00	0.15	59.99	60.19	60.28	60.38	60.57	1765	1.00
a[2]	60.14	0.00	0.17	59.83	60.04	60.14	60.25	60.46	1099	1.00
a[3]	60.56	0.00	0.15	60.27	60.47	60.56	60.66	60.86	1588	1.00
a[4]	60.61	0.00	0.16	60.32	60.51	60.61	60.71	60.92	1301	1.00

We see that the posterior means for the two SDs are $\hat{\sigma}_\alpha = 0.43$ and $\hat{\sigma}_\epsilon = 0.35$, which is a little smaller than the previous result although not radically different. We can also plot the posteriors in the same way as previously to find a slightly more concentrated distribution for σ_α . We are reassured that the results are not particularly sensitive to the choice of priors. We prefer the latter result since this is more consistent with reasonable assumptions about the data-generating mechanism.

Randomized Block Design: In Section 10.6, we analyzed some data on penicillin production that had treatments as fixed effects and blends (of the raw material) being random effects. We repeat this analysis here using STAN. We create STAN code in a file called `rbd.stan` in the R working directory:

```
data {
  int<lower=0> N;
  int<lower=0> Nt;
  int<lower=0> Nb;
  int<lower=1,upper=Nt> treat[N];
  int<lower=1,upper=Nb> blk[N];
  vector[N] y;
}
parameters {
  vector[Nb] eta;
  vector[Nt] trt;
  real<lower=0> sigmabl;
  real<lower=0> sigmaepsilon;
}
transformed parameters {
  vector[Nb] bld;
  vector[N] yhat;

  bld <- sigmabl * eta;

  for (i in 1:N)
    yhat[i] <- trt[treat[i]] + bld[blk[i]];
```

```

}
model {
  eta ~ normal(0, 1);

  y ~ normal(yhat, sigmaepsilon);
}

```

The STAN code is a generalization of the one-way ANOVA example. We load and put the data into a list consistent with the data declaration in the STAN program:

```

data(penicillin, package="faraway")
penidat <- list(N=20, Nt=4, Nb=5, y=penicillin$yield, treat=as.numeric
  ↪ (penicillin$treat), blk=as.numeric(penicillin$blend))

```

We break up the fitting process into the three component parts:

```

rt <- stanc(file="rbd.stan")
sm <- stan_model(stanc_ret = rt, verbose=FALSE)
fit <- sampling(sm, data=penidat)

```

The first step converts the STAN code to C++ and the second step compiles that code. We only need to do this once. We could save this and load it back in a future session like this:

```

save(sm, file="rbd.RData")
load("rbd.RData")

```

Since we often want to refit a model or try it with different data, this saves us some time by avoiding the repetition of the same steps. We must check the convergence of the MCMC. Trouble is most likely to appear in the block SD parameter, σ_b , so we check that:

```

traceplot(fit, pars="sigmabl", inc_warmup = FALSE)

```

The plot, shown in the first panel of Figure 12.3, reveals no problem. Checks of the other parameters are similarly uneventful. Now we study the summary output, selecting only the parts of immediate interest:

```

print(fit, pars=c("trt", "sigmabl", "sigmaepsilon", "bld"), probs=c
  ↪ (0.025, 0.5, 0.975))

```

```

4 chains, each with iter=2000; warmup=1000; thin=1;
post-warmup draws per chain=1000, total post-warmup draws=4000.

```

	mean	se_mean	sd	2.5%	50%	97.5%	n_eff	Rhat
trt[1]	83.97	0.35	3.74	76.87	83.81	92.06	114	1.03
trt[2]	85.03	0.30	3.58	77.46	85.01	92.39	139	1.03
trt[3]	89.07	0.37	3.75	81.81	88.99	98.79	105	1.04
trt[4]	86.01	0.33	3.54	78.98	85.88	95.16	118	1.03
sigmabl	5.35	0.41	3.99	0.68	4.27	17.70	95	1.03
sigmaepsilon	4.95	0.04	1.11	3.29	4.78	7.57	674	1.01
bld[1]	4.12	0.31	3.63	-2.11	3.93	11.75	138	1.03
bld[2]	-2.09	0.32	3.48	-11.73	-1.64	4.05	119	1.03
bld[3]	-0.59	0.31	3.34	-7.76	-0.50	6.10	115	1.04
bld[4]	1.30	0.33	3.42	-6.73	1.20	8.17	106	1.04
bld[5]	-2.77	0.37	3.67	-12.68	-2.40	3.69	101	1.04

The effective sample sizes are large enough so we are satisfied with the fit. The posterior means are comparable to the REML estimates obtained from the `lme4` fit.

We can plot the posterior densities for the two random SD parameters:

```

postsig <- rstan::extract(fit, pars=c("sigmabl", "sigmaepsilon"))
ref <- melt(postsig, value.name="yield")

```

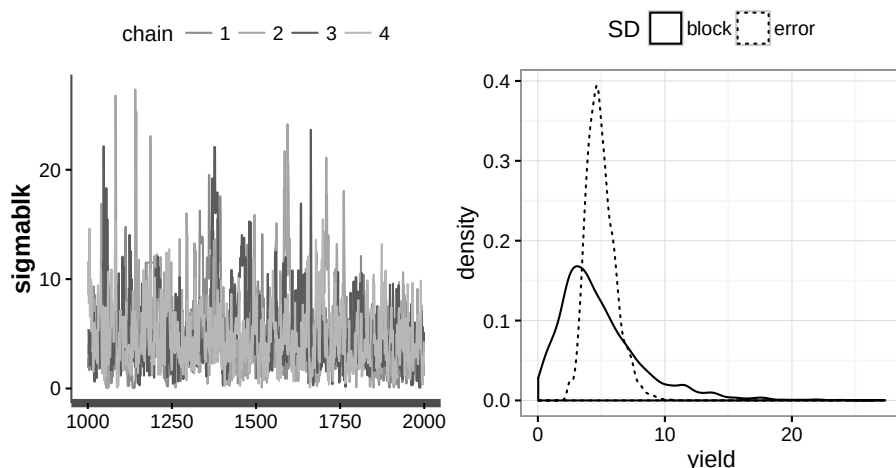


Figure 12.3 Diagnostic plot for the MCMC for σ_b is shown on the left. Posterior densities for the random SDs are shown on the right.

```
ggplot(data=ref, aes(x=yield, linetype=L1)) + geom_density() + scale_
  ↳ linetype(name="SD", labels=c("block", "error"))
```

We see in the second panel of Figure 12.3 that the error SD distribution is quite concentrated while the block SD is more diffuse. As with the previous example, it makes little sense to ask whether the blend SD is zero. The plot makes it clear that there probably is substantial variation between blends. We could ask whether the blend SD is less than some small value—one seems a reasonable limit in this example:

```
mean(postsig$sigmablck < 1)
[1] 0.0435
```

This confirms that there is a small chance that the blend SD is negligibly small but it seems much more likely that it is considerable. Given the relatively small effective sample size of 95 for this parameter, we would want more MCMC iterations to get a more accurate estimate if this was needed. We can plot the blend posterior distributions:

```
bldeff <- rstan::extract(fit, pars="bld")
rdf <- data.frame(yield=unlist(bldeff), blend=gl(5, 4000))
ggplot(data=rdf, aes(x=yield, linetype=blend)) + geom_density()
```

We see in the first panel of Figure 12.4 that although there is some separation in the distributions, it would be difficult to conclude that one was clearly better than another. The bumps in the tails of the densities are caused by the small sample size and could be removed with a larger number of MCMC iterations if desired.

We can construct the posteriors for the treatment effects also:

```
trteff <- rstan::extract(fit, pars="trt")
rdf <- data.frame(yield=unlist(trteff), treat=gl(4, 4000, labels=
  ↳ LETTERS[1:4]))
ggplot(data=rdf, aes(x=yield, linetype=treat)) + geom_density()
```

In the second panel of Figure 12.4 we see that there is considerable overlap between

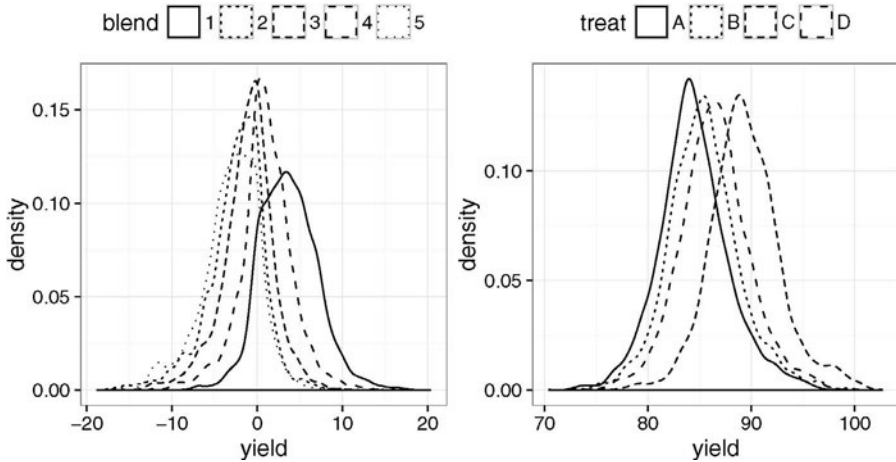


Figure 12.4 Posterior densities for the blend effects are shown on the left, while the treatment effects are shown on the right.

the distributions with A appearing worst and C best. We can estimate the probability that A is actually better than C as:

```
mean(trteff$trt[,1] > trteff$trt[,3])
[1] 0.05725
```

We conclude there is no clear difference between the treatments. If forced to choose, perhaps with information about the relative costs of the treatments, we could use the posterior distributions to make a rational decision.

It would also be reasonable to consider the treatments as a random effect. We could modify the STAN program to accommodate this and analyze accordingly.

STAN analyses of the other mixed effect models from previous chapters may be found in the online materials.

12.2 INLA

INLA stands for integrated nested Laplace approximation and was introduced by Rue et al. (2009). A Laplace approximation is a method for computing integrals of the form $\int \exp f(x) dx$. It requires only the maximum of f and the second derivative of $f(x)$ at that point. It can be surprisingly accurate given how little information is required. The INLA method builds on this idea and can be used for a wide class known as Gaussian Markov random field models. This class includes all the models considered in the previous two chapters. Because the method requires no simulation, it is much faster than MCMC-based approaches.

INLA is an R package that provides an interface to the INLA method. Visit <http://www.r-inla.org> for installation instructions as the package is not available from CRAN.

One-Way ANOVA: Let's use INLA to fit a model to the `pulp` data that we have

already explored with `lme4` in Chapter 10 and with STAN in Section 12.1. See Figure 10.1 for a plot of the data and Section 12.1 for a specification of the model.

We load the library and data before fitting the model.

```
library(INLA)
data(pulp, package="faraway")
formula <- bright ~ f(operator, model="iid")
result <- inla(formula, family="gaussian", data=pulp)
```

There is only the fixed effect intercept term which is included implicitly by default in the formula for the model. We specify how the random operator enters the model using the `f()` term. We expect no correlation in the observations for each operator so we set the `iid` option. We use the Gaussian family for the response when fitting the model with `inla()`. We examine the result and look at a somewhat-edited output:

```
summary(result)
```

Fixed effects:

	mean	sd	0.025quant	0.5quant	0.975quant	mode	kld
(Intercept)	60.4	0.0876	60.226	60.4	60.573	60.4	0

Random effects:

Name	Model
operator	IID model

Model hyperparameters:

	mean	sd	0.025quant
Precision for the Gaussian observations	6.895	2.131	3.494
Precision for operator	18377.553	18267.005	1205.289
	0.5quant	0.975quant	mode
Precision for the Gaussian observations	6.647	11.78	6.183
Precision for operator	12974.392	66416.77	3248.557

Expected number of effective parameters(std dev): 1.013(0.0132)

Number of equivalent replicates : 19.74

We see that the intercept of 60.4 is the same as in previous model fits. INLA uses the precision, which is the inverse of the variance, for its internal computations. We need to compute SDs for interpretative purposes. We have a posterior mean for $\hat{\sigma}_\varepsilon$ of $1/\sqrt{6.895} = 0.38$, but the precision for the operator term is very large and so the posterior mean for σ_α is close to zero. Even without our previous modeling experience with this data, we know from the plots of the data that we could not possibly be this confident about a conclusion of no variation between operators. Something is wrong.

The problem is the default priors chosen by INLA for the random effects precision terms. A gamma distribution with a very large variance is used. Past experience with LMMs reveals that this can sometimes lead to anomalous results. Indeed, this illustrates a general problem with so-called noninformative priors. We would like to have some default choice that will not have much effect on the outcome, but sometimes it will fail. One solution is to use a more informative prior that uses the weak information that we are likely to possess.

Let a random effect have SD σ and choose an exponential prior on σ such that a tail probability $P(\sigma > U) = \alpha$. We can pick a small value for α such as 0.01. Let's allow ourselves to look at the SD of the response and set $U = 3SD(Y)$. We are not constraining σ very much but we still allow a small probability that it could be even

three times larger than the response SD. This is an example of a penalized complexity prior as explained in Simpson et al. (2014).

```
sdres <- sd(pulp$bright)
pcprior <- list(prec = list(prior="pc.prec", param = c(3*sdres,0.01)))
formula <- bright ~ f(operator, model="iid", hyper = pcprior)
result <- inla(formula, family="gaussian", data=pulp)
result <- inla.hyperpar(result)
summary(result)
```

We specify $U = 3SD(Y)$ and $\alpha = 0.01$ in the penalized complexity prior. We use this prior for σ_α , but retain the default prior for σ_ϵ . The estimates for the posterior densities are improved by `inla.hyperpar`. The summary output shows precisions that are believable so we move onto a consideration of the posteriors.

Unlike MCMC programs, INLA does not just give you a sample from the posterior. It gives you the posterior density itself. Since we cannot easily plot the full multivariate posterior, we resort to looking at the marginals. The precisions are not very interpretable so we naturally want to convert it to the SD scale. Since we are transforming a density, the calculation is exact and can be achieved as follows:

```
sigmaalpha <- inla.tmarginal(function(x) 1/sqrt(exp(x)), result$
  ↪ internal.marginals.hyperpar[[2]])
sigmaepsilon <- inla.tmarginal(function(x) 1/sqrt(exp(x)), result$
  ↪ internal.marginals.hyperpar[[1]])
```

We can plot these as seen in the first panel of Figure 12.5:

```
ddf <- data.frame(rbind(sigmaalpha, sigmaepsilon), errterm=gl(2,1024,
  ↪ labels = c("alpha", "epsilon")))
ggplot(ddf, aes(x,y, linetype=errterm))+geom_line()+xlab("bright")+
  ↪ ylab("density")+xlim(0,2)
```

We see that the posteriors for the error and operator SDs are similar to those seen in Figure 12.2.

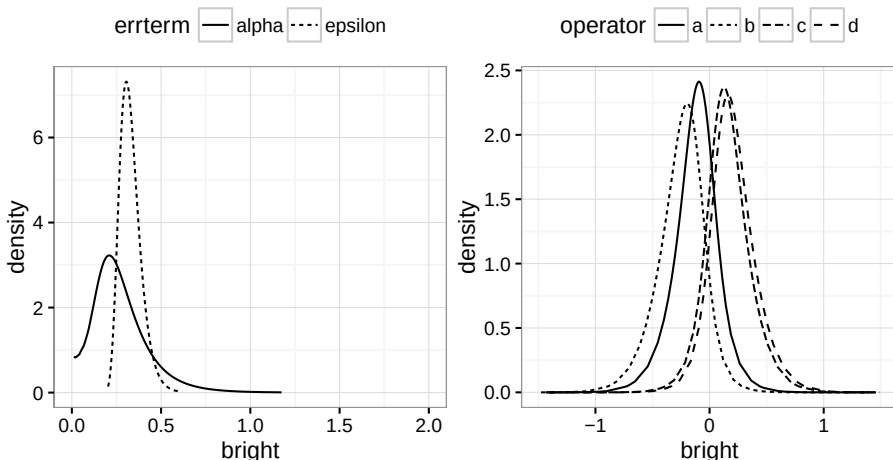


Figure 12.5 Posterior densities for the SDs are shown on the right. On the left, we see the posterior densities for the operator effects.

In the STAN analysis, we asked whether the operator SD was small. Specifically we computed $P(\sigma_\alpha < 0.1)$. We can compute this here by:

```
inla.pmarginal(0.1, sigmaalpha)
```

```
[1] 0.095447
```

The probability is still small but larger than that seen in the STAN-based analysis. Our prior for INLA put more weight on $\sigma = 0$, so this is not surprising.

We can plot the posteriors for the operator effects as seen in the second panel of Figure 12.5:

```
rdf <- do.call(rbind.data.frame, result$marginals.random$operator)
rdf <- cbind(operator=gl(4,nrow(rdf)/4,labels=letters[1:4]),rdf)
ggplot(rdf, aes(x=x,y=y,linetype=operator))+geom_line()+xlim(-1.5,
  ↪ 1.5)+xlab("bright")+ylab("density")
```

There is considerable overlap between the densities, meaning that it will be difficult to distinguish between specific operators.

Finally, we can obtain a numerical summary of the parameters of interest by:

```
restab <- sapply(result$marginals.fixed, function(x) inla.zmarginal(x,
  ↪ silent=TRUE))
restab <- cbind(restab, inla.zmarginal(sigmaalpha,silent=TRUE))
restab <- cbind(restab, inla.zmarginal(sigmaepsilon,silent=TRUE))
restab <- cbind(restab, sapply(result$marginals.random$operator,
  ↪ function(x) inla.zmarginal(x, silent=TRUE)))
colnames(restab) = c("mu", "alpha", "epsilon", levels(pulp$operator))
data.frame(restab)
```

	mu	alpha	epsilon	a	b	c	d
mean	60.4	0.24684	0.32115	-0.11008	-0.23533	0.15157	0.19329
sd	0.13238	0.11499	0.047074	0.14841	0.15769	0.1509	0.15398
quant0.025	60.03	0.041186	0.23559	-0.54174	-0.70209	-0.21423	-0.16563
quant0.25	60.297	0.16693	0.28606	-0.22994	-0.36867	0.037022	0.076544
quant0.5	60.399	0.24632	0.32084	-0.10922	-0.23292	0.1482	0.1894
quant0.75	60.501	0.34822	0.36309	0.0011632	-0.11752	0.27338	0.31963
quant0.975	60.768	0.66073	0.4694	0.2622	0.11642	0.59304	0.64648

Compare these posterior means to those obtained from the `lme4` analysis in Chapter 10 and in the earlier STAN analysis. The results are comparable but not the same.

Randomized Block Design: In Section 10.6, we analyzed some data on penicillin production that had treatments as fixed effects and blends (of the raw material) being random effects. Refer back to this section for a description of the model and plots of the data. We repeat this analysis here using INLA:

```
data(penicillin, package="faraway")
lmod <- lm(yield ~ treat, data=penicillin)
sdres <- sd(residuals(lmod))
pcprior <- list(prec = list(prior="pc.prec", param = c(3*sdres,0.01)))
formula <- yield ~ treat + f(blend, model="iid", hyper = pcprior)
result <- inla(formula, family="gaussian", data=penicillin)
result <- inla.hyperpar(result)
summary(result)
```

Use of the default prior for the blend precision gives rise to the same problem seen in the `pulp` analysis. We supply a more informative prior making use of the residual SD from the fixed effects only model. This SD will be bigger than the blend or error SD. The prior allows the possibility that the SD could be bigger still so we are doing little to constrain the outcome but ensuring a reasonable result. We transform the precisions to the SD scale:

```
sigmaalpha <- inla.tmarginal(function(x) 1/sqrt(exp(x)),result$
  ⇨ internal.marginals.hyperpar[[2]])
sigmaepsilon <- inla.tmarginal(function(x) 1/sqrt(exp(x)),result$
  ⇨ internal.marginals.hyperpar[[1]])
```

We produce a summary table of the posterior densities:

```
restab <- sapply(result$marginals.fixed, function(x) inla.zmarginal(x,
  ⇨ silent <- TRUE))
restab <- cbind(restab, inla.zmarginal(sigmaalpha,silent=TRUE))
restab <- cbind(restab, inla.zmarginal(sigmaepsilon,silent=TRUE))
colnames(restab) <- c("mu", "B-A", "C-A", "D-A", "alpha", "epsilon")
data.frame(restab)
```

	mu	B.A	C.A	D.A	alpha	epsilon
mean	84.028	0.96989	4.9565	1.9666	3.1441	4.2801
sd	2.0898	2.1297	2.1297	2.1297	1.4291	0.72534
quant0.025	78.76	-4.3684	-0.38568	-3.3725	0.48106	3.003
quant0.25	82.332	-0.7651	3.2211	0.2315	2.1311	3.7446
quant0.5	84.017	0.95897	4.9457	1.9557	3.1579	4.275
quant0.75	85.702	2.6811	6.6674	3.6777	4.3837	4.9327
quant0.975	89.28	6.2686	10.251	7.2646	7.908	6.6071

The posterior means are quite similar to the previous analyses. Particularly for asymmetric densities, the posterior mode is more analogous to a maximum likelihood estimate. These can be calculated for the two SDs as:

```
c(inla.mmarginal(sigmaalpha), inla.mmarginal(sigmaepsilon))
[1] 2.8331 4.0070
```

We examine the posterior densities of the SDs as seen in the first panel of Figure 12.6.

```
ddf <- data.frame(rbind(sigmaalpha,sigmaepsilon),errterm=gl(2,1024,
  ⇨ labels = c("alpha","epsilon")))
ggplot(ddf, aes(x,y, linetype=errterm))+geom_line()+xlab("yield")+ylab
  ⇨ ("density")+xlim(0,15)
```

We see that the error SD is more precisely specified than the block SD as is expected for such experiments. As before, we might ask whether there is much difference between the blends. We might express this by $P(\sigma_\alpha < 1)$. We can compute this as:

```
inla.pmarginal(1, sigmaalpha)
[1] 0.089164
```

We see that there is a small but significant chance that the blend SD is negligible.

We can also obtain the posterior densities for the treatment effects as seen in the second panel of Figure 12.6.

```
x <- seq(-15,15,length.out = 100)
rden <- sapply(result$marginals.fixed,function(y) inla.dmarginal(x, y)
  ⇨ )[,,-1]
ddf <- data.frame(yield=rep(x,3), density=as.vector(rden), treat=gl
  ⇨ (3,100, labels=c("B-A","C-A","D-A")))
ggplot(ddf, aes(x=yield, y=density, linetype=treat))+geom_line()
```

The standard parameterization of the fixed effects means that these represent differences from the the reference level, A. We can estimate the probability that A is actually better than C as:

```
inla.pmarginal(0,result$marginals.fixed$treatC)
[1] 0.033363
```

If we use the usual p -value reasoning that doubles this for a two-sided test, we obtain a value of about 6.6%. Given that the other differences are smaller, we find there is little evidence of a significant difference between the treatments. Given this is a Bayesian analysis, the usual justification for p -values does not apply, but we can

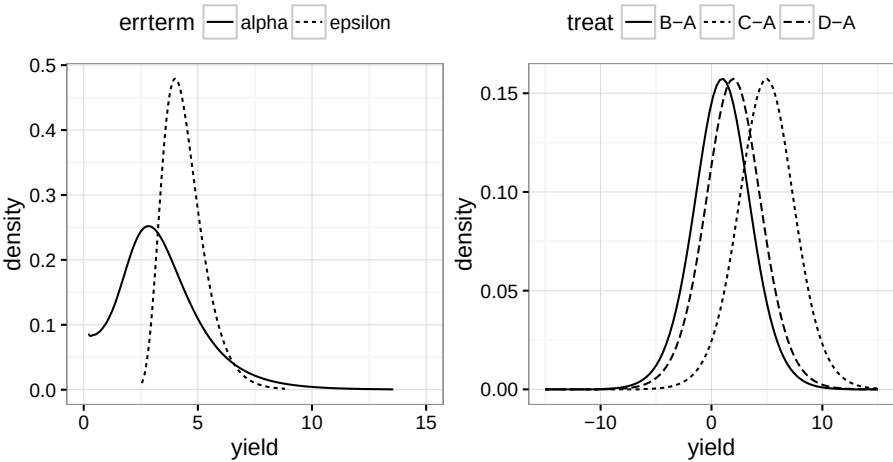


Figure 12.6 Posterior densities for SDs on the left and for the treatment effects on the right.

reasonably compute such quantities as summaries of the posterior densities if it is helpful.

12.3 Discussion

The maximum likelihood analysis of linear mixed models, demonstrated in Chapters 10 and 11, has several advantages. The models can be specified and fit with a single R command. The statistical hypothesis testing paradigm is widely accepted and may be required for the communication of some scientific research. The calculation of the p -values can be difficult, but is possible, even if simulation methods, such as the bootstrap, are required. Even so, problems may arise in fitting these models, particularly to larger datasets. Some types of valid questions cannot be answered in this mode of analysis.

The Bayesian approach offers a quite different way of analyzing this class of models. It offers several advantages in that we can use prior information to improve the inference and we can answer various relevant questions about the application in natural ways. There are some drawbacks. The models are more difficult to specify and require more programming knowledge, particularly when using STAN. The fitting process may fail in ways which are difficult to diagnose and rectify. The specification of reliable, so-called noninformative priors does not seem possible as failures producing unreasonable results are not uncommon. This requires us to think carefully about the specification of these priors. To the Bayesian, this is expected, but to others, this introduces an additional element of subjectivity which makes reaching convincing conclusions more difficult.

INLA beats STAN for speed, especially since it produces posterior densities not samples. For small datasets, the savings of a few minutes is not so important, but for larger datasets, the difference can be critical. STAN is more versatile as we are

able to program for a wider range of models than considered here. There is a general difficulty in validating a Bayesian analysis. Sometimes the outcome will be clearly flawed and we can make modifications. More worrisome are results which seem plausible, but are wrong. INLA produces approximations, but it is difficult to know when these are adequate. In principle, the MCMC analysis is essentially exact given a large enough run, but the simulation-based procedures used by STAN can fail in practice for various reasons that may be difficult to spot. For these reasons, a careful analyst is advised to use both INLA and STAN as a significant difference between the analyses will be revealing. Comparison with the `lme4`-based results provides a further check.

Exercises

1. The `denim` dataset concerns the amount of waste in material cutting for a jeans manufacturer due to five suppliers. See another question on this dataset in Chapter 10.
 - (a) Plot the data and comment.
 - (b) Fit the one-way ANOVA model using INLA using the default prior. Comment on the fit.
 - (c) Refit the model but with more informative priors. Make a density plot of the error and supplier SD posterior densities.
 - (d) Calculate summaries of the posteriors from the model fit.
 - (e) Report 95% credible intervals for the SDs using the summary output. Compute the posterior modes for the error and supplier SDs and compare these to the posterior means.
 - (f) Remove the two outliers from the data and repeat the analysis. Comment on any interesting differences.
2. Use the `denim` dataset again for this question but conduct the analysis using STAN.
 - (a) Fit the one-way ANOVA model using STAN with the default prior. Produce diagnostic plots for the three parameters: the mean and standard deviations of the supplier and error effects.
 - (b) Report the posterior mean, 95% credible intervals and effective sample size for the three parameters.
 - (c) Make a plot of the posterior densities of the supplier and error effects. Estimate the probability that the supplier SD is bigger than the error SD.
 - (d) Plot the posterior distributions of the five suppliers. Which supplier tends to produce the least waste and which the most? What is the probability that the best supplier is better than the worst supplier?
 - (e) A plot of the data reveals two obvious outliers. Repeat the analysis without these two points and report on any interesting differences with the full data.

- (f) Instead of removing the two outliers, change the error distribution from normal to t_3 . Repeat the analysis and indicate how this changes the conclusions. Discuss which approach to handling the outliers is best.
- 3. The `oatvar` dataset concerns a randomized block design comparing the yields of eight varieties of oats. The growing area was divided into five blocks, each planted with a single plot of each variety.
 - (a) Plot the data and comment.
 - (b) Use `INLA` to fit a linear model with variety as a fixed effect and block as a random effect using informative priors. Make plots of the posterior SD densities of the error and block.
 - (c) Compute a numerical summary of the posteriors. Which varieties show a significant difference from the first variety?
 - (d) Plot the densities of the variety effects.
 - (e) Consider a model with both variety and block as random effects. Use `INLA` to fit the model and construct a density plot of the three posterior SDs. How do these SDs compare?
- 4. Repeat question three using `STAN` in place of `INLA`.

This page intentionally left blank